PYTHON, AI DEVELOPMENT AND GITHUB COPILOT

INFOSYS SPRINGBOARD INTERNSHIP PROGRAM

AN INTERNSHIP PROJECT REPORT

Submitted by

SIVA ANKESH. N

(Reg No: 110725102239)

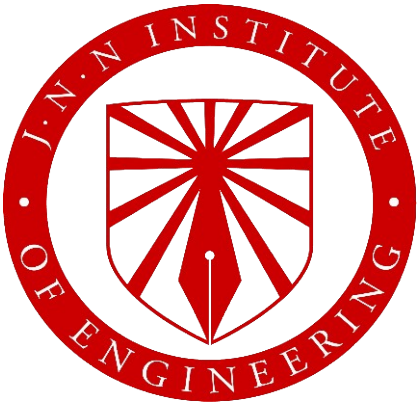
in partial fulfilment for the award of the degree of

of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE



J.N.N INSTITUTE OF ENGINEERING

(An Autonomous Institute, Affiliated to Anna University, Chennai)

AUGUST – 2026

VISION OF THE INSTITUTE:

Lead the transformation of engineering and management learning experience to educate the next generation of innovators and entrepreneurs who want to make the world a better place.

MISSION OF THE INSTITUTE:

IM1: To develop the required resources and infrastructure and to establish a conducive ambience for the teaching-learning process.

IM2: To nurture professional and ethical values in the students and to instil in them a spirit of innovation and entrepreneurship.

IM3: To encourage a desire for higher learning and research in the students and to equip them to face global challenges.

IM4: To provide opportunities for students to learn job-relevant skills to make them industry ready.

IM5: To interact with industries and other organizations to facilitate transfer of knowledge and know-how.

VISION OF THE DEPARTMENT:

To impart quality education, inculcate professionalism and enhance the problem-solving skills of the students in the domain of Artificial Intelligence & Data Science by applying recent technological tools and incorporating collaborative principles with a focus to make them industry ready.

MISSION OF THE DEPARTMENT:

DM1: To enhance the knowledge of the students with most recent advancements and refresh their insights in the field of Artificial Intelligence and Data Science.

DM2: To equip the students with strong fundamental concepts, analytical capability, programming and problem-solving skills.

DM3: To make the students industry ready and to enhance their employability through training, internships and real-time projects.

DM4: To guide the students to perform research on Artificial Intelligence and Data Science, with the aim to provide solutions to the problems of the industry.

PROGRAM OUTCOMES

S.NO	GRADUATE ATTRIBUTE	PROGRAM OUTCOMES
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

S.NO	PROGRAM EDUCATIONAL OBJECTIVES (PEO)
PEO1	To provide graduates with the proficiency to utilize the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volume of data.
PEO2	To enrich graduates with necessary technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
PEO3	To enable graduates to think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team.

PROGRAM SPECIFIC OBJECTIVES (PSOs)

S.NO	PROGRAM SPECIFIC OBJECTIVES (PSOs)
PSO1	Graduates should be able to evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.
PSO2	Graduates should be able to arrive at actionable Foresight, Insight, hindsight from data for solving business and engineering problems.
PSO3	Graduates should be able to create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems.

Department of Artificial Intelligence & Data Science

BONAFIDE CERTIFICATE

Certified that this internship project report **"PYTHON, AI DEVELOPMENT AND GITHUB COPILOT"** is the bonafide work of **"SIVA ANKESH. N" (Reg No: 110725102239)** who carried out the internship project work under my supervision in the Department of Artificial Intelligence and Data Science, J.N.N Institute of Engineering, Chennai.

Signature of the HOD with date	Signature of the Supervisor with date
Dr. P. Nagarajan Professor and Head, Department of Artificial Intelligence & Data Science, **J.N.N Institute of Engineering,** **Chennai**	**Mrs. M. Gayathridevi** Faculty Supervisor, Department of Artificial Intelligence & Data Science, **J.N.N Institute of Engineering,** **Chennai**

Certified that the above student has attended the viva voce examination held on .....

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to our beloved FOUNDER & CHAIRMAN Shri. S. JAYACHANDRAN B.Sc., B.L., and also heartfelt thanks to our VICE CHAIRMAN Mr. NAVEEN JAYACHANDRAN M.S., for providing the necessary facilities for the completion of my project.

I thank our PRINCIPAL Dr. G. MOHANBABU, Ph.D., for the encouragement provided with the congenial environment essential for me to complete the project.

I take this opportunity to express my gratitude to our HEAD OF THE DEPARTMENT Dr. P. NAGARAJAN and all other faculty members in the Department of ARTIFICIAL INTELLIGENCE AND DATA SCIENCE for giving me continuous encouragement for completing my project.

I extend my sincere thanks to my faculty supervisor, Mrs. M. Gayathridevi, for her guidance and support during the preparation of this internship report. I am also grateful to the Infosys Springboard learning platform and the course instructors of "AI & Python Development – Megaclass – 300+ Hands-on Projects" and "GitHub Copilot – The Complete Guide – 2026" for the structured learning content that made this internship possible.

Finally, I would like to thank my parents and friends for their constant support and encouragement throughout the completion of this internship and project report.

ABSTRACT

This internship report presents the learning and practical development activities completed in the areas of Python development, Artificial Intelligence and AI-assisted software development using GitHub Copilot through the Infosys Springboard learning environment. The report documents the progression from programming fundamentals and hands-on Python development toward Artificial Intelligence concepts and modern developer productivity tools.

The learning was verified through two course completion certificates: "AI & Python Development – Megaclass – 300+ Hands-on Projects", with a recorded duration of 58 total hours and completed on 23 August 2026, and "GitHub Copilot – The Complete Guide – 2026", with a recorded duration of 7 total hours and completed on 24 August 2026. Together these account for 65 hours of structured learning.

The report discusses the role of Python in AI and data-oriented development, the stages of a typical machine-learning workflow, and the use of GitHub Copilot for code generation, completion, explanation, debugging and documentation. A practical development workflow is described in which every AI-generated suggestion is reviewed, executed and tested before it is accepted. The report also records the skills acquired, the challenges faced and the approaches used to overcome them, and the future scope for further learning.

Overall, the internship strengthened programming fundamentals, built awareness of AI workflows and developed a disciplined approach in which AI assistance is treated as a support for learning and not as a substitute for understanding.

TABLE OF CONTENTS

CHAPTER NO	Title	Page
1	**INTRODUCTION**	10
	1.1. OVERVIEW	10
	1.2. INTERNSHIP DETAILS	10
	1.3. OBJECTIVES	11
	1.4. SCOPE OF THE PROJECT	11
	1.5. PROBLEM STATEMENT AND MOTIVATION	11
	1.6. ORGANISATION OF THE REPORT	12
	1.7. INTERNSHIP LEARNING PATH	12
2	**LITERATURE REVIEW**	14
	2.1. PYTHON FOR SOFTWARE AND AI DEVELOPMENT	14
	2.2. ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING	14
	2.3. AI-ASSISTED SOFTWARE DEVELOPMENT	14
	2.4. GITHUB COPILOT	14
	2.5. RESPONSIBLE USE OF AI CODING TOOLS	15
	2.6. PYTHON AI ECOSYSTEM AND DATA QUALITY	15
	2.7. TOOLS USED	15
3	**METHODOLOGY**	16
	3.1. LEARNING APPROACH	16
	3.2. PRACTICAL DEVELOPMENT WORKFLOW	16
	3.3. USE OF AI-ASSISTED DEVELOPMENT	16
	3.4. TESTING AND VALIDATION	17
	3.5. GENERAL MACHINE-LEARNING WORKFLOW	17
	3.6. ASSUMPTIONS	17
4	**IMPLEMENTATION**	18
	4.1. PYTHON DEVELOPMENT	18
	4.2. GITHUB COPILOT IN PRACTICE	19
	4.3. VALIDATION OF GENERATED CODE	20
	4.4. INTEGRATION OF SKILLS	20
5	**RESULTS AND DISCUSSION**	22
	5.1. KEY OUTCOMES AT A GLANCE	22
	5.2. SKILLS ACQUIRED	22
	5.3. LEARNING OUTCOMES	22
	5.4. CHALLENGES AND HOW THEY WERE ADDRESSED	23
	5.5. PROFESSIONAL RELEVANCE	23
	5.6. LIMITATIONS OF THE STUDY	23
6	**CONCLUSION AND FUTURE WORK**	24
	6.1. CONCLUSION	24
	6.2. OVERALL INTERNSHIP OUTCOME	24
	6.3. FUTURE WORK	24
	6.4. SKILLS DEVELOPED	24
	6.5. FUTURE ENHANCEMENT ROADMAP	25
	REFERENCES	25
	APPENDICES	**26**
	A. ADDITIONAL PRACTICE PROGRAMS	26
	B. GLOSSARY OF TERMS	27
	C. MAPPING TO PROGRAMME OUTCOMES	28
	D. REVIEW CHECKLIST AND REFLECTION	29
	CERTIFICATE EVIDENCE	30

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

The internship provided a practical introduction to Python programming, Artificial Intelligence concepts and AI-assisted software development. The learning began with Python fundamentals such as variables, data types, operators, conditional statements, loops, strings, lists, tuples, sets, dictionaries, functions, modules, file handling and exception handling. These concepts were supported by hands-on exercises so that programming logic could be understood through direct implementation.

As the learning progressed, the focus moved towards Artificial Intelligence concepts such as datasets, features, targets, training, testing and model evaluation. The final stage introduced GitHub Copilot, an AI-powered coding assistant, and explored how it can support code generation, completion, explanation, debugging and documentation inside a normal development workflow.

The report follows a simple workflow that was repeated throughout the internship: understand the requirement, plan the logic, write or request code assistance, inspect the generated result, run the program, test it, and document the final solution.

1.2 INTERNSHIP DETAILS

The details of the internship are summarised in the figure and tables below.

LEARNING PLATFORM	DOMAIN	DURATION	TOOLS USED
Infosys Springboard	**Python, AI Development and AI-Assisted Programming**	**August 2026 (65 learning hours)**	**Python, GitHub Copilot**

Figure 1.1: Internship at a glance

PARTICULARS	DETAILS
Intern Name	Siva Ankesh. N
Register Number	110725102239
Programme	B.Tech – Artificial Intelligence & Data Science
Institution	J.N.N Institute of Engineering, Chennai
Faculty Supervisor	Mrs. M. Gayathridevi
Learning Platform	Infosys Springboard
Domain	Python, AI Development and GitHub Copilot
Duration	August 2026 (course completions on 23.08.2026 and 24.08.2026)
Tools Used	Python, GitHub Copilot

Table 1.1: Internship details

COURSE	INSTRUCTOR	DURATION	COMPLETED ON
AI & Python Development – Megaclass – 300+ Hands-on Projects	School of AI	58 hours	23.08.2026
GitHub Copilot – The Complete Guide – 2026	Alex Dan	7 hours	24.08.2026

Table 1.2: Courses completed during the internship

1.3 OBJECTIVES

- To develop Python programming skills through hands-on exercises.
- To understand fundamental Artificial Intelligence and machine-learning concepts.
- To gain practical experience through hands-on programs and projects.
- To learn about AI-assisted software development and its responsible use.
- To understand and use GitHub Copilot for code generation, completion, explanation and debugging.
- To improve problem-solving, testing and debugging skills.
- To apply the concepts learned to practical development tasks.

1.4 SCOPE OF THE PROJECT

The scope of the internship covers Python development, introductory Artificial Intelligence concepts, hands-on programming, AI-assisted development with GitHub Copilot and the related practical skills of testing, debugging and documentation. The work demonstrates learning of the tools and workflows rather than the training of production machine-learning models. The same foundation can later be extended to larger Python applications, deeper machine learning and real-world AI projects.

1.5 PROBLEM STATEMENT AND MOTIVATION

Academic study provides theoretical knowledge, but students often find a gap between classroom learning and the practical, industry-oriented skills expected of a developer. Modern software development increasingly involves AI-assisted tools, yet such tools can produce code that looks correct but is incomplete, inefficient or wrong. A developer who lacks strong fundamentals cannot judge such output.

The problem addressed in this internship is therefore to build a solid base in Python and AI concepts and then learn to use GitHub Copilot in a controlled workflow, where every suggestion is reviewed, executed and tested. The motivation came from the need to become industry ready, which is also the aim of the department mission (DM3) and the institute mission (IM4).

1.6 ORGANISATION OF THE REPORT

The report is divided into six chapters. The table below gives a short map of what each chapter contains.

CHAPTER	TITLE	CONTENT
1	Introduction	Overview, internship details, objectives, scope, problem statement and report organisation.
2	Literature Review	Python, Artificial Intelligence, AI-assisted development, GitHub Copilot, responsible use and tools.
3	Methodology	Learning approach, development workflow, machine-learning workflow, testing and assumptions.
4	Implementation	Python programs, Copilot-assisted development, code validation and integration of skills.
5	Results and Discussion	Skills acquired, learning outcomes, challenges, professional relevance and limitations.
6	Conclusion and Future Work	Conclusion, skills developed, future roadmap and references.

Table 1.3: Organisation of the report

1.7 INTERNSHIP LEARNING PATH

The internship followed a step-by-step path in which each stage prepared the ground for the next one. The figure and table below summarise this progression from basic programming to AI-assisted development.

1	2	3	4	5
Python basics, operators, conditionals	Strings, lists, tuples, sets, dictionaries	Loops, functions, modules, file and exception handling	AI and machine-learning concepts	GitHub Copilot and AI-assisted development

Figure 1.2: Learning path followed during the internship

STAGE	TOPICS COVERED	OUTCOME
1	Variables, data types, operators, conditional statements	Understanding of basic program logic and control flow.
2	Strings, lists, tuples, sets, dictionaries	Ability to store, access and organise different types of data.
3	Loops, functions, modules, file handling, exception handling	Ability to write reusable and robust programs.
4	Datasets, features, targets, training, testing, evaluation	Understanding of the basic AI/ML workflow.
5	GitHub Copilot: generation, completion, explanation, debugging, documentation	Ability to use AI suggestions as a starting point and validate them.

Table 1.4: Learning path followed during the internship

CHAPTER 2
LITERATURE REVIEW

2.1 PYTHON FOR SOFTWARE AND AI DEVELOPMENT

Python is a high-level programming language widely used for software development, automation, data processing and Artificial Intelligence. Its simple syntax allows beginners to focus on problem solving, while its ecosystem provides libraries for numerical computing, data analysis, visualisation and machine learning.

Core language concepts. The learning treated Python as the foundation for practical programming. Core topics included variables, data types, operators, conditional statements, loops, strings, lists, tuples, sets, dictionaries, functions, modules, file handling and exception handling.

Data structures. Lists were studied as ordered, changeable collections, tuples as ordered collections that cannot be changed, sets as collections of unique elements, and dictionaries as key-value stores. Understanding these structures is essential when reading code generated by an AI assistant.

2.2 ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Artificial Intelligence refers to computational methods that enable systems to perform tasks that normally require aspects of human intelligence, such as pattern recognition, prediction, reasoning and language processing. Machine Learning is a major area of AI in which algorithms learn patterns from data and use those patterns to make predictions or decisions.

The internship introduced AI concepts at a practical level. Important ideas include datasets, features, targets, training, testing, model evaluation and the importance of data quality. These concepts provide the foundation required for later AI development.

2.3 AI-ASSISTED SOFTWARE DEVELOPMENT

AI-assisted software development uses machine-learning models to support programmers while they write and maintain software. Such tools can suggest code, complete partially written statements, explain unfamiliar code, generate documentation and help identify possible errors.

AI assistance does not remove the need for programming knowledge. Developers remain responsible for checking correctness, security, maintainability, licensing considerations and suitability of the generated output.

2.4 GITHUB COPILOT

GitHub Copilot is an AI-powered coding assistant designed to provide contextual coding suggestions. The internship focused on understanding how the tool can assist during code writing and software-development activities. Its main capabilities are:

- Code completion based on the surrounding programming context.
- Generation of code from natural-language descriptions.

- Assistance with repetitive programming tasks.
- Explanation of existing code and programming constructs.
- Suggestions for debugging and possible improvements.
- Support for writing comments and documentation.

2.5 RESPONSIBLE USE OF AI CODING TOOLS

AI-generated code should be treated as a suggestion rather than an automatically correct solution. A responsible workflow includes understanding the requirement, generating or writing code, reviewing the output, executing tests, checking edge cases and improving the implementation where required.

Important practices include avoiding the submission of sensitive information to external AI services, reviewing generated dependencies, checking security-sensitive code carefully and maintaining human ownership of the final technical decision.

2.6 PYTHON AI ECOSYSTEM AND DATA QUALITY

Python provides a broad ecosystem for AI development. NumPy supports numerical operations, Pandas supports structured data manipulation, Matplotlib and related tools support visualisation, and Scikit-learn provides many classical machine-learning algorithms and evaluation utilities.

AI systems are strongly influenced by the quality of the data used for development. Missing values, inconsistent formats, irrelevant features and biased data can reduce the reliability of results. Therefore, data preparation and validation are important parts of any AI workflow.

2.7 TOOLS USED

The internship relied on a small set of tools, each with a clear role. The table below summarises them.

TOOL	ROLE IN THE INTERNSHIP	KEY FEATURES USED
Python	Main programming language	Variables, control flow, functions, file handling and exception handling.
GitHub Copilot	AI-assisted coding	Code generation, completion, explanation, debugging support and documentation.
Infosys Springboard	Learning environment	Structured access to the course content and practice material.
Udemy courses	Course content and certification	AI & Python Megaclass (58 hours) and GitHub Copilot guide (7 hours).

Table 2.1: Tools used and their roles

CHAPTER 3
METHODOLOGY

3.1 LEARNING APPROACH

The internship followed a progressive learning approach. First, programming fundamentals were revised and practised. Next, Artificial Intelligence concepts were studied at an introductory level. Finally, GitHub Copilot was used to explore AI-assisted programming workflows. The six phases are shown below.

PHASE	ACTIVITY	OUTPUT
1	Concept learning	Understanding of Python and AI fundamentals.
2	Programming practice	Working Python programs and exercises.
3	AI learning	Understanding of the basic AI/ML workflow.
4	Copilot exploration	Experience with AI-generated suggestions.
5	Review and testing	Validated and improved code.
6	Reflection	Identification of skills, challenges and future learning areas.

Table 3.1: Phases of the learning approach

3.2 PRACTICAL DEVELOPMENT WORKFLOW

A repeated development workflow was followed for practical exercises: understand the requirement, plan the logic, write or request code assistance, inspect the generated result, run the program, test normal and edge cases, correct errors and document the final solution.

This workflow ensured that AI assistance was used as a productivity and learning aid rather than as a replacement for understanding the program.

3.3 USE OF AI-ASSISTED DEVELOPMENT

At every step, GitHub Copilot supported the work while the responsibility for correctness remained with the student, as shown below.

STEP	GITHUB COPILOT SUPPORT	STUDENT RESPONSIBILITY
Requirement	Suggest possible implementation approaches.	Understand the problem and select the appropriate approach.
Coding	Generate or complete code.	Review syntax and logic.
Explanation	Explain functions or code blocks.	Confirm the explanation and learn the concept.
Debugging	Suggest possible causes and fixes.	Reproduce the error and verify the fix.
Documentation	Draft comments or descriptions.	Check technical accuracy and clarity.
Testing	Suggest test cases.	Execute tests and interpret results.

Table 3.2: Division of work between Copilot and the student

3.4 TESTING AND VALIDATION

Testing was treated as an essential step in AI-assisted development. A generated program was not considered complete merely because it looked syntactically correct. The program needed to execute successfully and produce results consistent with the intended requirement. The following checks were applied:

- Check expected inputs and outputs.
- Test boundary and unusual inputs.
- Read error messages and trace the cause.
- Compare generated logic with the problem statement.
- Simplify or rewrite code when generated code is unnecessarily complex.

3.5 GENERAL MACHINE-LEARNING WORKFLOW

The AI portion of the learning focused on the general workflow used in data-driven applications, from data collection to prediction.

STAGE	PURPOSE
Data collection	Obtain relevant observations for the problem.
Data preparation	Clean and transform data into a usable form.
Feature selection	Identify useful input variables.
Training	Allow a model to learn patterns from training data.
Evaluation	Measure how well the model performs on unseen data.
Prediction	Use the trained model to produce an output for new input.

Table 3.3: General machine-learning workflow

3.6 ASSUMPTIONS

The work is learning-oriented. Code examples are short illustrative programs written to practise the concepts, and no claim is made that any model was trained on a real-world dataset.

CHAPTER 4
IMPLEMENTATION

4.1 PYTHON DEVELOPMENT

Python development formed the foundation of the practical learning activities. Programs were used to practise basic syntax and progressively move toward reusable functions and structured solutions.

4.1.1 Basic Programming

```
name = input('Enter your name: ')
print('Hello', name)
```

The example demonstrates input, variable assignment and output. Simple exercises of this type help build confidence before moving to larger programs.

4.1.2 Conditional Logic

```
marks = int(input('Enter marks: '))
if marks >= 50:
    print('Pass')
else:
    print('Fail')
```

Conditional statements allow programs to make decisions based on input values. Understanding conditions is important when reviewing AI-generated code because a developer must be able to determine whether the suggested logic matches the requirement.

4.1.3 Lists and Loops

```
numbers = [10, 20, 30, 40]
for number in numbers:
    print(number)
```

Lists and loops are common programming constructs and frequently appear in generated code. Practising them manually helped improve the ability to read and validate Copilot suggestions.

4.1.4 Functions

```
def calculate_total(price, quantity):
    return price * quantity

print(calculate_total(100, 3))
```

Functions support modular programming by grouping reusable logic. AI coding assistants can generate functions quickly, but the developer still needs to verify parameter names, return values and edge cases.

4.1.5 Exception Handling

```
try:
    value = int(input('Enter a number: '))
    print(value)
except ValueError:
    print('Please enter a valid number.')
```

Exception handling improves the robustness of programs by providing controlled responses when invalid input or runtime problems occur.

4.1.6 File and Data Handling

Practical Python learning also included reading and processing information from files. File handling is important in AI workflows because data often needs to be loaded, cleaned and transformed before analysis or model development.

```
with open('data.txt', 'r') as file:
    content = file.read()
print(content)
```

4.2 GITHUB COPILOT IN PRACTICE

GitHub Copilot was studied as an AI-assisted programming tool. The practical learning focused on using natural-language instructions and surrounding code context to obtain useful suggestions.

4.2.1 Code Generation

A developer can describe a programming requirement in a comment or prompt and use the generated suggestion as a starting point. For example, the following comment describes a requirement:

```
# Write a function that returns the largest value in a list
```

The generated implementation must still be reviewed. The student checks whether the function handles empty lists, correct data types and other relevant edge cases. A reviewed version of such a function is shown below as an illustration of the review process.

```
def largest_value(values):
    if not values:
        raise ValueError('The list is empty.')
    largest = values[0]
    for v in values[1:]:
        if v > largest:
            largest = v
    return largest
```

4.2.2 Code Completion

When partially written code is available, Copilot can suggest the next lines based on the surrounding context. This can reduce repetitive typing and allow the programmer to focus more on the structure of the solution.

4.2.3 Code Explanation

An AI assistant can help explain unfamiliar functions or code blocks. This is particularly useful during learning because the student can compare the explanation with their own understanding and then verify it by running small experiments.

4.2.4 Debugging Support

When a program produces an error, Copilot can suggest possible causes and corrections. The debugging process remains iterative: reproduce the problem, understand the error message, inspect the suggested fix, apply it and run the program again.

4.2.5 Documentation

AI assistance can also help draft comments, function descriptions and documentation. Generated documentation should be checked for accuracy because it may describe behaviour that the program does not actually implement.

4.3 VALIDATION OF GENERATED CODE

Applying the testing approach of Section 3.4 to the function in Section 4.2.1, the following test cases were used to confirm that the reviewed code behaves as required.

TEST CASE	INPUT	EXPECTED RESULT
Normal list	[3, 9, 2]	9
Negative numbers	[-5, -2, -9]	-2
Single element	[7]	7
Empty list	[]	ValueError raised with a clear message

Table 4.1: Test cases for the reviewed function

4.4 INTEGRATION OF SKILLS

The major outcome of the practical learning was the integration of conventional programming with AI assistance. Python provided the programming foundation, AI concepts provided the application context and GitHub Copilot provided an additional development aid.

SKILL	HOW IT WAS INTEGRATED
Python	Used for programming exercises and implementation practice.
AI concepts	Used to understand data-driven problem-solving workflows.
Problem solving	Used to convert requirements into program logic.
GitHub Copilot	Used for code generation, completion, explanation and debugging support.
Testing	Used to validate AI-generated or manually written code.
Documentation	Used to record learning and explain technical work.

Table 4.2: Integration of skills

CHAPTER 5
RESULTS AND DISCUSSION

5.1 KEY OUTCOMES AT A GLANCE

2 courses completed	**65** total learning hours	**300+** hands-on projects in the Python and AI course	**6** Copilot capabilities studied

Figure 5.1: Key outcomes at a glance

5.2 SKILLS ACQUIRED

The internship strengthened both technical and professional skills. The most important result was an improved understanding of how programming fundamentals and AI-assisted tools can be combined in a controlled development workflow.

- Improved Python programming fundamentals.
- Better understanding of Artificial Intelligence and machine-learning terminology.
- Practical familiarity with AI-assisted code generation and completion.
- Improved ability to read, explain and modify existing code.
- Improved debugging and testing habits.
- Greater awareness of the limitations of generated code.
- Improved technical documentation and communication skills.

5.3 LEARNING OUTCOMES

LEARNING OUTCOME	DESCRIPTION
Programming	Able to write and understand Python programs using core language constructs.
AI awareness	Able to describe the basic stages of an AI/ML workflow.
AI-assisted coding	Able to use Copilot suggestions as a starting point for development.
Code review	Able to inspect generated code for correctness and relevance.
Debugging	Able to use error messages and systematic testing to identify problems.
Responsible use	Understands that AI output requires human validation.

Table 5.1: Learning outcomes

5.4 CHALLENGES AND HOW THEY WERE ADDRESSED

CHALLENGE	APPROACH USED
Understanding generated code	Break the code into smaller sections and study each part.
Incorrect or incomplete suggestions	Rewrite the prompt, provide clearer context and verify the output by execution.
Debugging generated code	Read the error message, reproduce the problem and test the proposed correction.
Over-reliance on AI	Attempt the logic independently before using Copilot for assistance.
Complex generated solutions	Simplify the code and retain only the logic required by the problem.
Learning unfamiliar concepts	Use explanations, documentation and small experiments to confirm understanding.

Table 5.2: Challenges and solutions

5.5 PROFESSIONAL RELEVANCE

AI-assisted development is relevant to modern software engineering because developers increasingly work with tools that can automate repetitive coding tasks. The internship experience provides a foundation for using such tools productively while maintaining responsibility for code quality, security and correctness.

The experience is also relevant to future roles involving Python, Artificial Intelligence, data science and software development. The ability to understand requirements, write code, test solutions and communicate technical decisions remains important even when AI tools are available.

5.6 LIMITATIONS OF THE STUDY

- The internship was learning-oriented, so no production-scale AI system was built.
- The Copilot examples are short exercises and may not reflect work on large codebases.
- The evidence of completion consists of two course certificates; the report does not claim results beyond them.
- Machine-learning topics were studied at an introductory level and model training in depth is left for future study.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The internship provided a practical foundation in Python programming, Artificial Intelligence and AI-assisted software development. The work moved from programming fundamentals to the use of GitHub Copilot for code generation, completion, explanation, documentation and debugging support.

A key learning from the internship was that AI coding tools are most useful when combined with human understanding and validation. Generated code can accelerate development, but it must be reviewed, tested and adapted to the actual requirement.

6.2 OVERALL INTERNSHIP OUTCOME

Overall, the internship improved technical confidence, programming practice, awareness of AI workflows and familiarity with modern development tools. It also encouraged a disciplined approach in which AI assistance is treated as a support mechanism rather than a substitute for learning, and it provided a useful bridge between academic study and practical software development.

6.3 FUTURE WORK

- Develop larger Python applications using modular and object-oriented programming.
- Study machine learning in greater depth, including model training and evaluation.
- Explore deep learning frameworks and neural-network development.
- Build practical AI projects using real-world datasets.
- Explore retrieval-augmented generation and other modern generative-AI techniques.
- Learn software testing, Git workflows, APIs and deployment practices.
- Use GitHub Copilot for larger projects while applying code review and security practices.

6.4 SKILLS DEVELOPED

AREA	SKILLS GAINED
Technical	Python programming, functions and modules, file and exception handling, AI/ML workflow concepts and AI-assisted coding with GitHub Copilot.
Analytical	Reviewing generated code, debugging from error messages and designing test cases.
Professional	Documenting technical work, communicating decisions and using AI tools responsibly.

Table 6.1: Skills developed during the internship

6.5 FUTURE ENHANCEMENT ROADMAP

The future work listed in Section 6.3 can be organised into phases so that learning grows step by step.

PHASE	ENHANCEMENT	EXPECTED BENEFIT
1	Object-oriented Python and larger applications	Better structure and reuse of code.
2	Machine learning with Scikit-learn on real datasets	Hands-on model training and evaluation.
3	Git, testing and APIs	Professional development practice.
4	Deep learning and generative AI	Exposure to modern AI techniques.
5	Copilot on larger projects with code review	Faster, safer development.

Table 6.2: Roadmap for future enhancement

6.6 REFERENCES

[1] Infosys Springboard learning platform: https://infyspringboard.onwingspan.com/

[2] School of AI. AI & Python Development – Megaclass – 300+ Hands-on Projects. Course completion certificate, Udemy, completed by Siva Ankesh, 23 August 2026.

[3] Alex Dan. GitHub Copilot – The Complete Guide – 2026. Course completion certificate, Udemy, completed by Siva Ankesh, 24 August 2026.

[4] Python Software Foundation. Python documentation. https://docs.python.org/3/

[5] GitHub. GitHub Copilot documentation and learning resources.

[6] Scikit-learn developers. Scikit-learn documentation: user guide and API reference.

APPENDIX A
ADDITIONAL PRACTICE PROGRAMS

Besides the programs described in Chapter 4, further short programs were practised on the data structures, modules and file handling listed in Chapter 2. Each program was first written manually and then compared with the suggestion produced by GitHub Copilot, following the workflow of Section 3.2. The programs below are illustrative practice exercises.

A.1 DICTIONARIES

```
marks = {'Maths': 78, 'Physics': 85, 'Chemistry': 69}
total = 0
for subject, score in marks.items():
    total += score
print('Average:', total / len(marks))
```

A dictionary stores values against keys. The program loops over the key-value pairs and prints the average mark (77.33). While reviewing a Copilot suggestion for this task, the division by `len(marks)` was checked, and the case of an empty dictionary, which would cause a division-by-zero error, was noted.

A.2 SETS AND STRINGS

```
items = [1, 2, 2, 3, 3, 3]
print(sorted(set(items)))      # [1, 2, 3]

word = 'level'
print(word == word[::-1])     # True
```

A set keeps only unique elements, so converting a list to a set removes duplicates. The second example uses string slicing to test whether a word reads the same in both directions.

A.3 LISTS, LOOPS AND CONDITIONS

```
numbers = [3, 8, 12, 7, 10]
count = 0
for n in numbers:
    if n % 2 == 0:
        count += 1
print('Even numbers:', count)   # 3
```

This program combines a loop with a condition to count the even numbers in a list.

A.4 MODULES, FILES AND EXCEPTIONS

```
import math
print(math.sqrt(49))           # 7.0

try:
    with open('data.txt', 'r') as f:
        lines = f.readlines()
    print('Lines read:', len(lines))
except FileNotFoundError:
    print('The file was not found.')
```

Modules allow ready-made functionality to be reused. The file example combines file handling with exception handling so that a missing file produces a controlled message instead of a program crash.

A.5 TEST CASES FOR THE ADDITIONAL PROGRAMS

The testing approach of Section 3.4 was applied to the programs above. Expected results were worked out by hand before the programs were run.

PROGRAM	INPUT	EXPECTED RESULT
Average of marks	Maths 78, Physics 85, Chemistry 69	77.33 (approximately)
Remove duplicates	[1, 2, 2, 3, 3, 3]	[1, 2, 3]
Palindrome check	level	True
Palindrome check	python	False
Count even numbers	[3, 8, 12, 7, 10]	3
Square root	49	7.0
File reading	data.txt not present	Message: The file was not found.

Table A.1: Test cases for the additional practice programs

APPENDIX B
GLOSSARY OF TERMS

The following terms are used throughout this report.

TERM	MEANING
Dataset	A collection of observations used to build or evaluate a model.
Feature	An input variable that the model uses to learn a pattern.
Target	The value that a model is expected to predict.
Training	The stage in which a model learns patterns from training data.
Testing	Checking a model on data it has not seen during training.
Model evaluation	Measuring how well a model performs on unseen data.
Prompt	A natural-language instruction or comment given to an AI tool.
Code completion	A suggestion for the next lines of code based on the surrounding context.
Module	A Python file or library whose functions can be imported and reused.
Exception	A runtime error that can be caught and handled by the program.

Table B.1: Glossary of terms

APPENDIX C
MAPPING OF INTERNSHIP OUTCOMES TO PROGRAMME OUTCOMES

The outcomes of the internship contribute to the programme outcomes (PO), programme educational objectives (PEO) and programme specific objectives (PSO) of the department, as listed on pages 4 and 5 of this report.

C.1 MAPPING TO PROGRAMME OUTCOMES AND PSO

MAPPING	INTERNSHIP ACTIVITY	CONTRIBUTION
PO1	Python fundamentals and data structures	Application of programming fundamentals to the solution of problems.
PO2	Reviewing and debugging generated code	Analysing a problem and reaching a verified conclusion.
PO3	Converting requirements into program logic	Designing simple solutions that meet a stated need.
PO5	Use of Python and GitHub Copilot	Selecting and applying a modern tool while understanding its limitations.
PO8	Responsible use of AI coding tools	Awareness of ethics, security and accountability for the final code.
PO10	Documentation and this report	Writing clear technical documentation and reports.
PO12	Self-paced online courses	Independent learning in a changing technology area.
PSO3	AI/ML workflow and practical tools	Using AI concepts together with practical industrial tools.

Table C.1: Mapping of internship outcomes to PO and PSO

C.2 MAPPING TO PEO AND MISSION STATEMENTS

MAPPING	CONTRIBUTION
PEO1	Use of programming, AI and data-science fundamentals to build data-oriented solutions.
PEO3	Logical thinking and lifelong learning through independent completion of online courses.
DM2	Strengthening of programming and problem-solving skills.
DM3 / IM4	Industry readiness through training in job-relevant tools such as GitHub Copilot.

Table C.2: Mapping to PEO and mission statements

APPENDIX D
REVIEW CHECKLIST AND REFLECTION

D.1 CHECKLIST FOR REVIEWING AI-GENERATED CODE

The checklist below summarises the questions asked before any Copilot suggestion was accepted.

CHECK	QUESTION ASKED
Requirement	Does the code solve the stated problem and nothing else?
Correctness	Does it run and give the expected output for normal inputs?
Edge cases	What happens with empty, negative, very large or invalid input?
Clarity	Can every line be explained in the student's own words?
Simplicity	Can the code be shortened or simplified without changing the result?
Security	Does it use unsafe input handling, unknown dependencies or sensitive data?
Documentation	Do the comments describe what the code really does?

Table D.1: Review checklist for generated code

D.2 GUIDELINES FOR WRITING PROMPTS

- State the task clearly in one sentence, for example the function name, its input and its output.
- Give the surrounding context, such as variable names and the data type of the input.
- Mention special conditions, such as how empty input should be handled.
- If the suggestion is wrong, rewrite the prompt with more detail instead of accepting it.

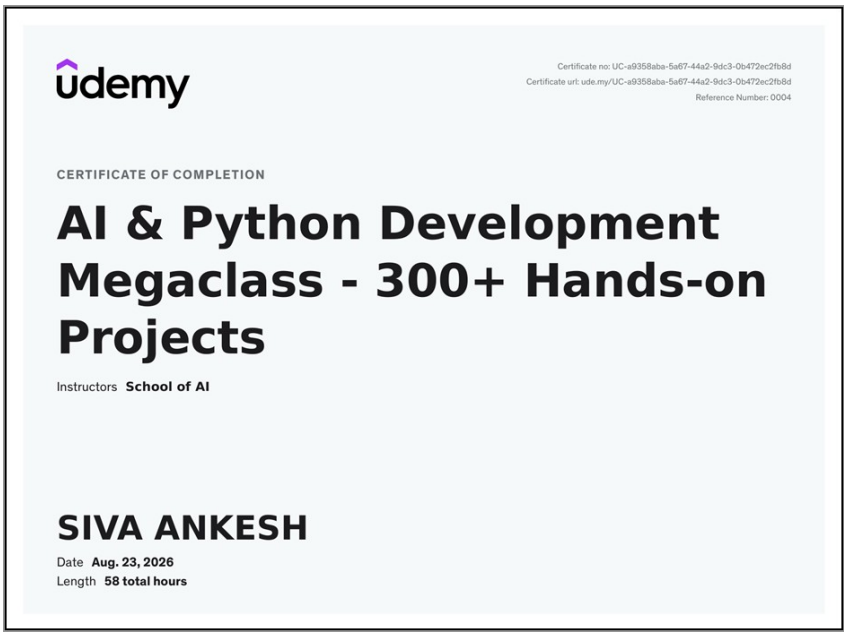
D.3 PERSONAL REFLECTION

The most valuable lesson of the internship was that writing code and understanding code are different skills. Copilot could produce a working-looking function in seconds, but only knowledge of conditions, loops and data structures made it possible to judge whether that function was correct. The habit of writing the logic first, requesting assistance second and testing last is the practice that will be carried into future projects, and it directly supports the industry-readiness aim stated in DM3 and IM4.

CERTIFICATE EVIDENCE

The following certificate evidence documents support the learning activities described in this report.

AI & Python Development – Megaclass – 300+ Hands-on Projects


udemy

Certificate no: UC-a9358aba-5a67-44a2-9dc3-0b472ec2fb8d
Certificate url: ude.my/UC-a9358aba-5a67-44a2-9dc3-0b472ec2fb8d
Reference Number: 0004

CERTIFICATE OF COMPLETION

AI & Python Development Megaclass - 300+ Hands-on Projects

Instructors School of AI

SIVA ANKESH

Date Aug. 23, 2026
Length 58 total hours

GitHub Copilot – The Complete Guide – 2026


udemy

Certificate no: UC-17b2010c-0843-4fca-babb-1b86f77cbd9f
Certificate url: ude.my/UC-17b2010c-0843-4fca-babb-1b86f77cbd9f
Reference Number: 0004

CERTIFICATE OF COMPLETION

GitHub Copilot - The Complete Guide - 2026

Instructors Alex Dan • 300K+ Students

SIVA ANKESH

Date Aug. 24, 2026
Length 7 total hours